

발행 요청 상태 추적 단건 발행 오케스트레이션

IssuanceRequest · IssuerService · issueOne() 파이프라인
TxStatus · MintStatus · IssuanceStatus 완전 이해

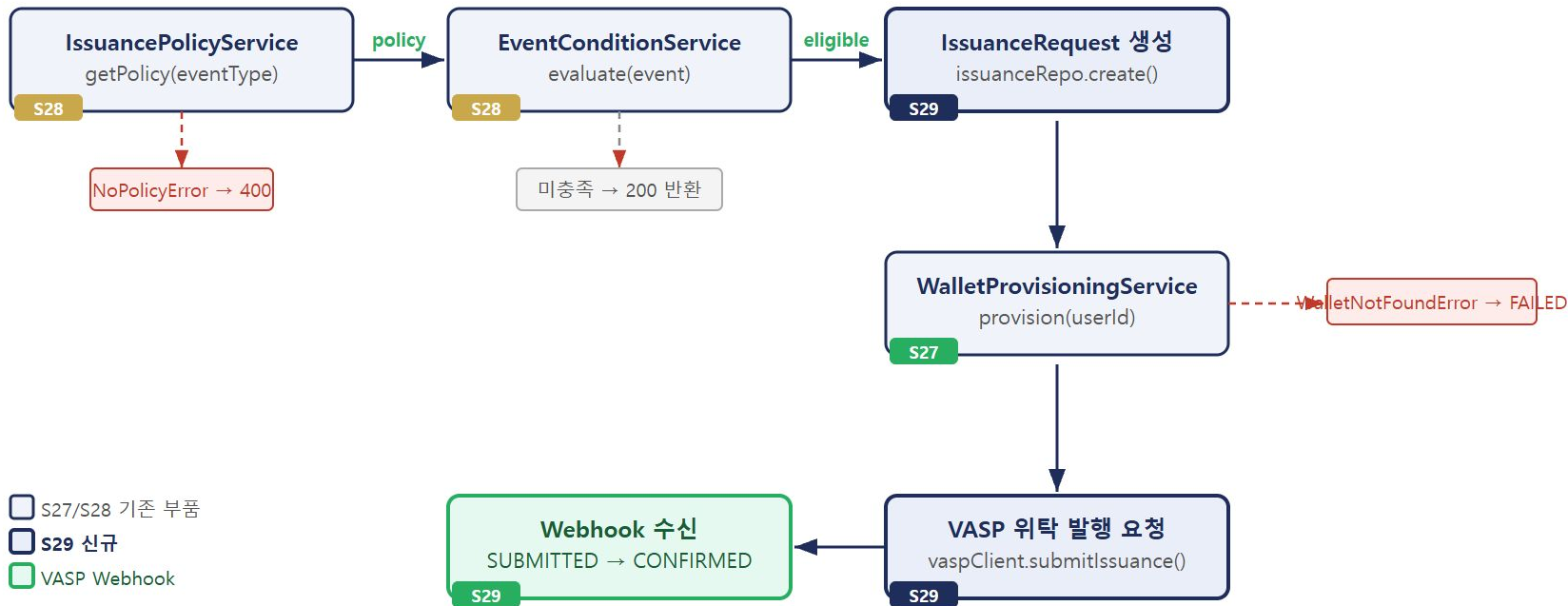
IssuanceStatus

IssuerService

VASP Webhook

S27~S28에서 만든 부품이 여기서 연결된다

IssuerService — 오케스트레이터. 직접 DB를 건드리지 않는다.



IssuanceRequest — 발행 요청 상태 엔티티

NFT 발행은 VASP에 위탁하는 순간 비등기가 된다 — 상태 추적이 필요하다

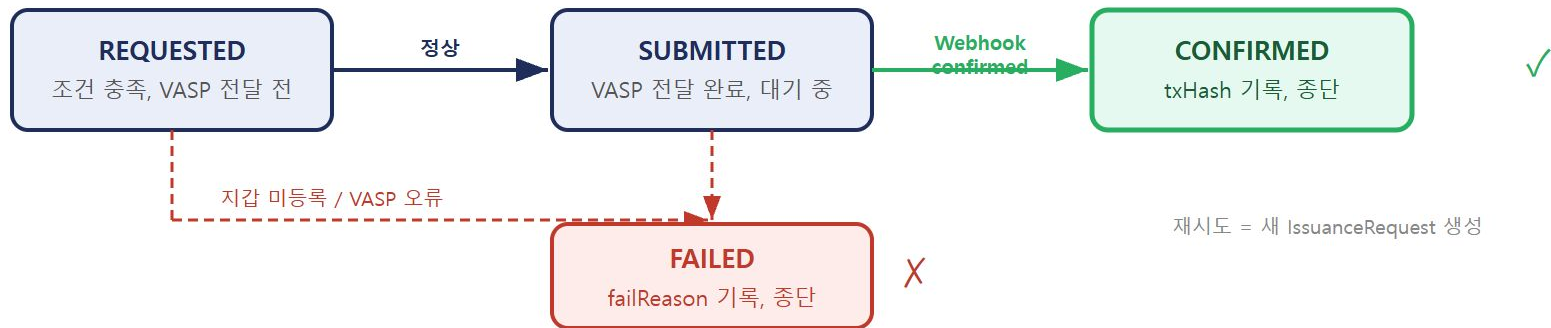
```
export type IssuanceStatus =  
  'REQUESTED' // 조건 통과, VASP 전달 전  
  | 'SUBMITTED' // VASP 전달 완료, 대기  
  | 'CONFIRMED' // Webhook 수신, 종단  
  | 'FAILED'; // 어느 단계든 실패, 종단
```

```
export interface IssuanceRequest {  
  id: string; // UUID  
  userId: string;  
  eventType: string; // 'WALK_GOAL_MET'  
  tokenId: bigint; // 생성 시 즉시 확정  
  amount: number;  
  walletAddr: string | null; // 지갑 조회 전 null  
  status: IssuanceStatus;  
  txHash: string | null; // CONFIRMED 후 기록  
  failReason: string | null;  
  createdAt: Date;  
  updatedAt: Date;  
}
```

mint_requests(M4)와 다른 점 · tokenId가 생성 시 즉시 확정 (M4는 CONFIRMED 이후) · walletAddr-amount 포함 · blockNumber-retryCount 없음

IssuanceRequest — 상태 흐름

4개 상태 — VASP Webhook 기준으로 전이한다



상태	의미	누가 전이	DB 변화
REQUESTED	조건 충족, VASP 전달 전	IssuerService.issueOne()	INSERT, walletAddr 채워짐
SUBMITTED	VASP 전달 완료, 대기	IssuerService.issueOne()	status 갱신
CONFIRMED	Webhook 수신, 중단	WebhookHandler	txHash 채워짐
FAILED	파이프라인 임의 단계 실패, 중단	IssuerService / Webhook	failReason 채워짐

선행 개념 정리 — TxStatus / MintStatus / IssuanceStatus

(1 / 9)

왜 Status가 세 개인가 — 한 사건을 세 관점에서 본다

비즈니스 레이어

IssuerService · IssuanceStatus · issuance_request

관심사: 이 발행 요청이 완료됐는가? 어떤 이벤트에서 왔는가? · 상태: REQUESTED → SUBMITTED → CONFIRMED / FAILED

원장 레이어

LedgerService · MintStatus · mint_requests

관심사: DB 원장이 체인 상태와 일치하는가? 감사 추적 가능한가? · 상태: REQUESTED → SUBMITTED → MINED → CONFIRMED → FINALIZED / FAILED / REORGED

체인 TX 레이어

TxStateMachineService · TxStatus · tx_attempts

관심사: 블록체인 TX가 지금 어디 있는가? mempool? 블록? REORG? · 상태: REQUESTED → SUBMITTED → PENDING → MINED → CONFIRMED → FINALIZED / FAILED / REORGED

위로 올라갈수록 상태가 단순해진다 — 하위 레이어가 세부사항을 캡슐화하기 때문이다

선행 개념 정리 (2 / 9) TxStatus

블록체인 TX 상태 — M3에서 배운 것 (@kyobo/vasp 패키지)

무엇을 추적하나

블록체인 위에서 TX가 지금 어느 단계에 있는가

누가 관리하나

TxStateMachineService (@kyobo/vasp 패키지, 학생이 구현하지 않음)

DB 저장이 필요하다

`findPendingOlderThan(minutes)` 메서드 때문. 서버 재시작 후에도 오래된 PENDING TX를 찾아 재처리해야 하므로 반드시 DB에 기록 필요. 실습은 InMemoryRepo 사용(교육 편의상).

mint_requests(M4)와 다른 테이블

운영에서는 `tx_attempts` 같은 별도 테이블. M3 MintRequest와 M4 MintRequest는 컬럼이 다르다.

```
// TxRepository 인터페이스 (운영 = PostgreSQL 구현)
interface TxRepository {
    save(req: MintRequest): Promise<void>;
    findById(id: string): Promise<MintRequest | null>;
    updateStatus(
        id: string, status: TxStatus,
        extra?: Partial<MintRequest>
    ): Promise<void>;
    // ↓ 이 메서드 때문에 DB 저장 필수
    findPendingOlderThan(
        minutes: number
    ): Promise<MintRequest[]>;
}
```

```
-- tx_attempts 테이블 (운영 환경)
CREATE TABLE tx_attempts (
    id          UUID          PRIMARY KEY,
    user_id     VARCHAR(64),
    token_id    NUMERIC(78),
    amount      BIGINT,
    status      VARCHAR(16),  -- TxStatus
    tx_hash     VARCHAR(66),
    retry_count INT,
    created_at  TIMESTAMPTZ
);
```


선행 개념 정리 (3 / 9) TxStatus — 8개 상태

TxStateMachineService가 전이 — 학생은 사용만 한다

상태	의미	누가 전이시키나	DB 변화
REQUESTED	발행 요청 생성됨, VASP 전송 전	IssuerService	tx_attempts INSERT
SUBMITTED	VASP에 전송됨, TX hash 아직 없음	TxStateMachineService	tx_hash 미확정
PENDING	TX 전송됨, mempool 대기 중	TxStateMachineService	status 갱신
MINED	블록에 포함됨, finality 미확보	TxStateMachineService (콜백/풀링)	block_number 채워짐
CONFIRMED	충분한 블록 확인 완료	TxStateMachineService	status 갱신
FINALIZED	PoS 2/3+ validator 동의, 절대 불변 — 중단	TxStateMachineService	status 갱신
FAILED	REVERT 또는 최종 실패 — 중단	TxStateMachineService	error_msg 채워짐
REORGED	MINED 후 블록 재편으로 소실, 재처리 대기	TxStateMachineService	status 갱신

PENDING이 있는 이유 · mempool 대기 중 gas bump-timeout 처리가 필요하다 · MintStatus-IssuanceStatus에는 없다

선행 개념 정리 (4 / 9) MintStatus

DB 원장 상태 — M4에서 배운 것 (LedgerService + mint_requests)

무엇을 추적하나

mint_requests 테이블 레코드의 현재 상태

누가 관리하나

LedgerService (학생이 M4 S23~S24에서 직접 구현)

TxStatus와의 결정적 차이

PENDING(mempool) 없음 — LedgerService는 mempool 상태에 관심 없다.
"TX가 블록에 들어갔는가"부터 관심.

TxStateMachineService가 다리 역할

체인 이벤트 감지 → 내부 TxStatus 갱신 → LedgerService 호출 → mint_requests 갱신

```
CREATE TABLE mint_requests (  
  request_id  UUID          PRIMARY KEY,  
  user_id     VARCHAR(64)   NOT NULL,  
  policy_id   VARCHAR(64)   NOT NULL,  
  status      VARCHAR(16)   DEFAULT 'REQUESTED',  
  tx_hash     VARCHAR(66),  
  token_id    NUMERIC(78),   -- CONFIRMED 후 채워짐  
  block_number BIGINT,       -- MINED 후 채워짐  
  retry_count INT           DEFAULT 0,  
  error_msg   TEXT,  
  created_at  TIMESTAMPTZ   DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ   DEFAULT NOW()  
);
```

issuance_request와 다른 점

token_id가 CONFIRMED 이후 채워짐 (issuance_request는 생성 시 즉시) ·
walletAddr.amount 없음 · blockNumber.retryCount 있음

선행 개념 정리 (5 / 9) MintStatus — 7개 상태

LedgerService가 전이 — 학생이 M4에서 직접 구현한 것

상태	의미	누가 전이시키나	DB 변화
REQUESTED	발행 요청 생성됨	LedgerService.createMintRequest()	mint_requests INSERT
SUBMITTED	VASP에 TX 제출, tx_hash 확보	TxStateMachineService → LedgerService	tx_hash 채워짐
MINED	블록에 포함됨, finality 미확보	TxStateMachineService → LedgerService	block_number 채워짐
CONFIRMED	원장 확정 허용	TxStateMachineService → LedgerService	token_id 채워짐
FINALIZED	PoS finality 확보, 절대 불변 — 종단	TxStateMachineService → LedgerService	status 갱신
FAILED	최종 실패 — 종단	TxStateMachineService → LedgerService	error_msg 채워짐
REORGED	블록 재편으로 소실, 재채굴 대기	TxStateMachineService → LedgerService	status 갱신

MINED가 있는 이유 · ReconcileService(S25)가 "MINED이지만 CONFIRMED 미달" 요청을 찾아 체인과 대조한다. MINED ↔ CONFIRMED 구분이 없으면 조회 불가.

선행 개념 정리 (6 / 9) IssuanceStatus

비즈니스 발행 요청 상태 — M5 S29, 이번 세션 (IssuerService + issuance_request)

무엇을 추적하나

발행 요청이 오케스트레이션 파이프라인 어느 단계에 있는가

누가 관리하나

IssuerService (이번 세션에서 구현)

MINED / FINALIZED / REORGED가 없는 이유

Phase 1(VASP 위탁)에서 체인 세부 상태는 VASP가 처리. Webhook으로 "confirmed" 또는 "failed" 두 가지만 수신. 비즈니스는 결과만 받으면 된다.

IssuanceStatus CONFIRMED = MintStatus CONFIRMED + FINALIZED 함수
VASP가 'confirmed' Webhook을 보내는 시점 = 내부적으로 이미 FINALIZED 수준

```
CREATE TABLE issuance_request (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     VARCHAR(64)   NOT NULL,  
  event_type  VARCHAR(64)   NOT NULL,  
  token_id    NUMERIC(78)   NOT NULL, -- 생성 시 즉시 확정  
  amount      INT           NOT NULL DEFAULT 1,  
  wallet_addr VARCHAR(42),   -- 지갑 조회 후 채움  
  status      VARCHAR(16)   NOT NULL DEFAULT 'REQUESTED'  
  CHECK (status IN (  
    'REQUESTED', 'SUBMITTED',  
    'CONFIRMED', 'FAILED')),  
  tx_hash     VARCHAR(66),   -- CONFIRMED 후 채움  
  fail_reason TEXT,  
  created_at  TIMESTAMPTZ   DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ   DEFAULT NOW()  
);
```

선행 개념 정리 (7 / 9) IssuanceStatus — 4개 상태

IssuerService · WebhookHandler가 전이 — 이번 세션에서 구현

상태	의미	누가 전이시키나	DB 변화
REQUESTED	조건 충족, 지갑 조회 전 — walletAddr=null	IssuerService.issueOne() ③ 단계	issuance_request INSERT
SUBMITTED	VASP에 발행 요청 전달 완료, Webhook 대기	IssuerService.issueOne() ⑤ 단계	status 갱신
CONFIRMED	Webhook 수신, 온체인 TX 확정 — 중단	WebhookHandler	tx_hash 채워짐
FAILED	파이프라인 임의 단계 실패 — 중단	IssuerService / WebhookHandler	fail_reason 채워짐

FAILED 범위 — MintStatus보다 넓다

지갑 미등록

WalletNotFoundError
→ FAILED (체인 도달 전)

VASP 오류

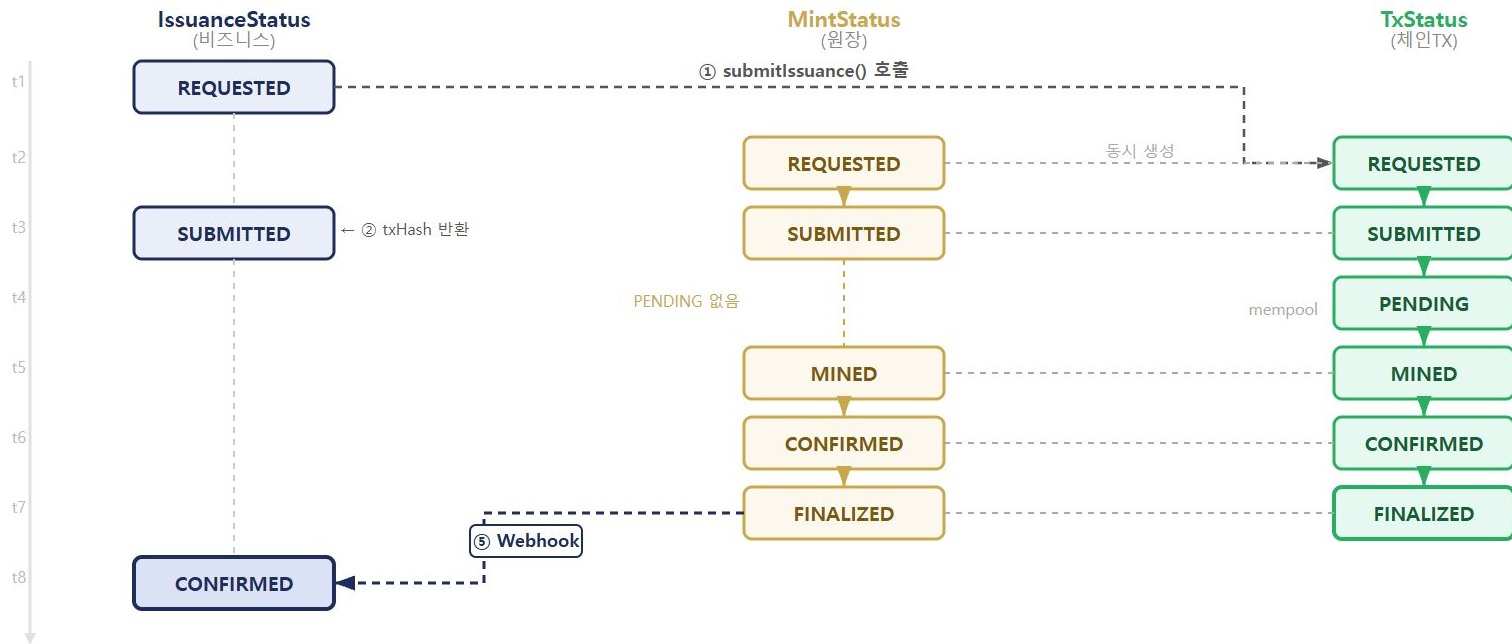
submitIssuance() 실패
→ FAILED (TX 미생성)

체인 실패

Webhook { status:'failed' }
→ FAILED (REVERT 등)

선행 개념 정리 (8 / 9) 세 Status를 한 흐름으로

시간 순서로 보기 — 레이어 간 영향 관계



선행 개념 정리 (9 / 9) 전체 상태 대응표

같은 이름이라도 레이어마다 의미가 다르다

TxStatus	MintStatus	IssuanceStatus	발생 시점 / 차이
REQUESTED	REQUESTED	REQUESTED	발행 요청 생성. IssuanceStatus는 지갑 주소 조회 전(walletAddr=null). MintStatus-TxStatus는 지갑 확보 후.
SUBMITTED	SUBMITTED	SUBMITTED	VASP 전달. TxStatus = TX hash 미확보. MintStatus = tx_hash 채워짐. IssuanceStatus = VASP 접수만 확인.
PENDING	없음 (mempool 관심 없음)	없음	mempool 대기 — 체인 TX 레이어만 추적
MINED	MINED	없음 (VASP가 추상화)	블록 포함. ReconcileService가 MINED↔CONFIRMED 구분 필요 → MintStatus에는 있음.
CONFIRMED	CONFIRMED	CONFIRMED	TxStatus/MintStatus = 아직 FINALIZED 전 중간 단계. IssuanceStatus = Webhook 'confirmed' 수신 시점 (= FINALIZED 수준).
FINALIZED	FINALIZED	(둘을 흡수)	PoS finality. IssuanceStatus CONFIRMED에 흡수됨.
REORGED	REORGED	없음 (VASP가 재처리)	블록 재편 소실. VASP가 재처리 후 결과만 Webhook으로 통보.
FAILED	FAILED	FAILED	TxStatus/MintStatus = 체인 실패만. IssuanceStatus = 지갑 미등록·VASP 오류·체인 실패 모두 포함.

IssuanceRequest 리포지토리 인터페이스

IssuerService는 이 인터페이스에만 의존한다 — 운영: Postgres / 테스트: InMemory

```
export interface IssuanceRequestRepository {  
  // 신규 발행 요청 생성  
  create(  
    req: Omit<IssuanceRequest, 'id' | 'createdAt' | 'updatedAt'>  
  ): Promise<IssuanceRequest>;  
  
  // ID로 단건 조회  
  findById(id: string): Promise<IssuanceRequest | null>;  
  
  // 상태 전이 + 추가 필드 업데이트  
  updateStatus(  
    id: string,  
    status: IssuanceStatus,  
    extra?: {  
      txHash?: string;  
      walletAddr?: string;  
      failReason?: string;  
    },  
  ): Promise<void>;  
}
```

인터페이스만 정의하면 된다

운영에서는 PgIssuanceRequestRepository

테스트에서는 makeInMemoryIssuanceRepo()

updateStatus 불변 조건

CONFIRMED 또는 FAILED 상태에서 전이 시도 → 에러

상태 전이 불가: CONFIRMED → REQUESTED

setWalletAddr 별도 메서드 고려

지갑 주소 업데이트는 상태 전이가 아닌 데이터 보정.

updateStatus(id, 'REQUESTED', { walletAddr }) 는
동일 상태 전이 → 가드 로직과 충돌 가능

IssuerService — 의존 관계

의존성 목록이 곧 서비스의 책임 목록이다

```
export class IssuerService {  
  constructor(private readonly deps: {  
    policyService:      IssuancePolicyService;  
    conditionService:   EventConditionService;  
    walletProvisioning: WalletProvisioningService;  
    issuanceRepo:       IssuanceRequestRepository;  
    vaspClient:          IVaspIssuanceClient;  
  }) {}  
}
```

의존성	역할	위치
policyService	tokenId:amount 조회	S28
conditionService	조건 충족 여부 판단	S28
walletProvisioning	사용자 지갑 주소 조회	S27
issuanceRepo	발행 요청 상태 기록	S29
vaspClient	VASP에 발행 위탁	S29

IssuerService는 직접 DB를 건드리지 않는다

모든 의존성이 인터페이스다. EVM → XRPL로 체인이 바뀌어도 IssuerService 코드는 변경 없다.

IVaspIssuanceClient 인터페이스

VASP 위탁 발행 — 비동기 응답 모델

```
export interface IVaspIssuanceClient {  
  submitIssuance(params: {  
    requestId: string;  
    walletAddr: string;  
    tokenId: bigint;  
    amount: number;  
  }): Promise<{  
    submitted: boolean;  
    vaspRefId: string;  
  }>;  
}
```

비동기 응답 모델

`submitIssuance()` 는 "요청을 접수했다"는 응답만 반환.
체인 확정 결과는 VASP → Webhook으로 별도 수신.

VASP 위탁 흐름

- ① submitIssuance() 호출 → VASP 접수
- ② IssuanceStatus: REQUESTED → SUBMITTED
- ③ VASP 내부에서 TX 서명·브로드캐스트
- ④ 체인 확정 후 Webhook 수신
- ⑤ IssuanceStatus: SUBMITTED → CONFIRMED

인터페이스에만 의존하는 이유

교보생명 VASP(월렛원)가 바뀌어도 `IVaspIssuanceClient` 구현체만 교체하면 된다. IssuerService 코드는 변경 없음.

issueOne() — 6단계 파이프라인 (1 / 4)

함수 선언부 · ① 발행 정책 조회

```
async issueOne(event: {
  userId:    string;
  eventType: string;
  data:      Record<string, unknown>;
}): Promise<{ requestId: string; eligible: boolean }> {
  const { userId, eventType, data } = event;

  // ① 발행 정책 조회
  const policy = await this.deps.policyService.getPolicy(eventType);
  // NoPolicyError 발생 시 상위로 전파 → 400
  // policy: { tokenId, amount, validFrom, validTo }
```

getPolicy()가 실패하면 · NoPolicyError throw → issueOne() 밖으로 전파 → 컨트롤러에서 400 응답 · issuance_request 레코드 생성 안 됨

policyService.getPolicy()는 DB를 읽는다 (issuance_policy 테이블). IssuerService는 직접 DB 접근하지 않고 policyService를 통해 간접 접근.

issueOne() — 6단계 파이프라인 (2 / 4)

② 조건 판단 · ③ 발행 요청 생성 (REQUESTED)

```
// ② 조건 판단
const condition = await this.deps.conditionService.evaluate({ userId, eventType, data });
if (!condition.eligible) {
  return { requestId: '', eligible: false };
  // 조건 미충족 → 발행 요청 자체를 생성하지 않는다
}

// ③ 발행 요청 생성 (REQUESTED)
const req = await this.deps.issuanceRepo.create({
  userId, eventType,
  tokenId: policy.tokenId, // 정책에서 가져온 값 - 즉시 확정
  amount: policy.amount,
  walletAddr: null, // 지갑 조회 전 null
  status: 'REQUESTED',
  txHash: null,
  failReason: null,
});
```

② eligible: false이면 ③ 실행 안 함

조건 미충족은 에러가 아니라 정상 흐름. 레코드가 없어야 맞다. 재처리 대상 이 아님을 명확히.

③ tokenId가 생성 시 확정되는 이유

정책 조회 결과가 이미 있다. MintStatus(mint_requests)와 달리 CONFIRMED까지 기다릴 필요 없음.

issueOne() — 6단계 파이프라인 (3 / 4)

④ 지갑 주소 조회

```
// ㉔ 지갑 주소 조회
let walletAddr: string;
try {
  const result = await this.deps.walletProvisioning.provision(userId, 'EXTERNAL');
  walletAddr = result.walletAddress;
  // walletAddr 보장 — 상태 전이 없음, 데이터 업데이트만
  await this.deps.issuanceRepo.setWalletAddr(req.id, walletAddr);
} catch (err) {
  await this.deps.issuanceRepo.updateStatus(req.id, 'FAILED', {
    failReason: (err as Error).message,
  });
  throw err;
  // WalletNotFoundError → 상위(Worker)가 DLQ로 이동
}
```

setWalletAddr() 별도 메서드

지갑 주소 업데이트는 상태 전이가 아닌 데이터 보장. `updateStatus(id, 'REQUESTED', { walletAddr })` 는 동일 상태 전이로 가드에서 에러 발생 가능.

지갑 조회 실패 → FAILED + throw

`issuance_request`에 FAILED를 기록하고 에러를 다시 던진다. 어느 단계에서 실패했는지 DB에서 즉시 파악 가능.

issueOne() — 6단계 파이프라인 (4 / 4)

⑤ VASP 위탁 발행 요청 · ⑥ SUBMITTED 상태로 반환

```
// ⑤ VASP 위탁 발행 요청
try {
  await this.deps.vaspClient.submitIssuance({
    requestId: req.id,
    walletAddr,
    tokenId: policy.tokenId,
    amount: policy.amount,
  });
  await this.deps.issuanceRepo.updateStatus(req.id, 'SUBMITTED');
} catch (err) {
  await this.deps.issuanceRepo.updateStatus(req.id, 'FAILED', {
    failReason: (err as Error).message,
  });
  throw err;
}

// ⑥ SUBMITTED 상태로 반환 - 체인 확인은 Webhook에서 처리
return { requestId: req.id, eligible: true };
}
```

issueOne()은 CONFIRMED를 직접 설정하지 않는다 · VASP에 요청을 보내고 SUBMITTED를 기록한 뒤 즉시 반환. 체인 확인은 나중에 Webhook 핸들러가 처리.

조건 미충족 시 REQUESTED 레코드를 만들지 않는 이유

② 단계에서 `eligible: false`이면 `issuanceRepo.create()` 호출 없이 즉시 반환

❌ 레코드를 만들면

- DB에 의미 없는 FAILED 레코드 누적
- "재처리 대상"과 "조건 미충족"을 구분하기 어려움
- 운영 모니터링에서 노이즈 발생
- `issuance_request`가 "발행 시도 이력"이 아닌 "이벤트 수신 이력"이 되어 버림

✓ 레코드를 만들지 않으면

- `issuance_request` 존재 = "발행이 시도되었다"는 의미
- 없음 = "조건 미충족, 시도 안 함"으로 명확
- FAILED 레코드 = 모두 재처리 검토 대상
- 조회 쿼리가 단순해짐

조건 미충족은 에러가 아니다 · 정상 흐름의 일부다 · 발행 요청이 존재한다는 것 = "발행이 시도되었다"는 의미 · 시도하지 않은 경우는 레코드가 없어야 맞다

상태 전이 불변 조건

온체인 TX는 되돌릴 수 없다 — CONFIRMED·FAILED는 중단 상태

전이 불가

CONFIRMED → REQUESTED 불가

CONFIRMED → FAILED 불가

FAILED → 어디든 불가

재발행이 필요하면 새 **IssuanceRequest**를 생성한다. 기존 레코드는 영구 보존.

```
async updateStatus(  
  id: string, newStatus: IssuanceStatus, ...  
) {  
  const current = await this.findById(id);  
  if (  
    current?.status === 'CONFIRMED' ||  
    current?.status === 'FAILED'  
  ) {  
    throw new Error(  
      `상태 전이 불가: ${current.status} → ${newStatus}`  
    );  
  }  
  // ... UPDATE 실행  
}
```

왜 CONFIRMED가 중단인가

온체인 TX가 확정된 이후 DB 상태를 되돌리면 실제 체인 상태와 불일치가 발생한다. 감사 로그 위반이기도 하다.

VASP Webhook — CONFIRMED 처리

issueOne()에서 하지 않는다 — Webhook 핸들러가 별도로 처리한다

```
// WebhookController (강의 범위 외, 개념만)
async handleVaspWebhook(payload: {
  requestId: string;
  txHash:    string;
  status:    'confirmed' | 'failed';
}) {
  if (payload.status === 'confirmed') {
    await this.issuanceRepo.updateStatus(
      payload.requestId, 'CONFIRMED',
      { txHash: payload.txHash }
    );
  } else {
    await this.issuanceRepo.updateStatus(
      payload.requestId, 'FAILED',
      { failReason: 'VASP reported failure' }
    );
  }
}
```

왜 issueOne()에서 CONFIRMED를 설정하지 않는가

체인 확인은 비동기다. issueOne()이 반환한 뒤 수십 초~수 분 후에 Webhook이 도착한다. 두 경로(요청·확인)를 분리해야 비동기 흐름이 명확해진다.

두 경로의 역할 분리

issueOne() → REQUESTED, SUBMITTED, FAILED 설정

WebhookHandler → CONFIRMED, FAILED 설정

CONFIRMED는 반드시 Webhook 경로에서만

먹등성 — 동일 이벤트 재수신 시 (1 / 2)

전략 1 — (userId, eventType, tokenId) 조합으로 중복 체크

```
// issueOne() 내부 @ 조건 판단 직후
const existing = await this.deps.issuanceRepo.findPending(
  userId, eventType, policy.tokenId
);
if (existing) {
  return { requestId: existing.id, eligible: true };
  // 이미 진행 중인 요청이 있으면 새 요청을 만들지 않는다
}
```

findPending 조회 조건

- user_id = userId
- event_type = eventType
- token_id = policy.tokenId
- status IN ('REQUESTED', 'SUBMITTED')

왜 CONFIRMED는 조건에 없나

CONFIRMED는 완료된 요청. 같은 이벤트로 두 번 발행하면 안 되므로 CONFIRMED도 체크해야 하는지는 비즈니스 정책에 따라 다름.

전략 1의 장점

기존 코드 변경 없이 적용 가능. 단, 동시성 문제(race condition) 가능성 있음.

먹등성 — 동일 이벤트 재수신 시 (2 / 2)

전략 2 — 이벤트 ID를 IssuanceRequest에 기록해 UNIQUE 제약

```
-- issuance_request 테이블에 event_id 컬럼 추가
ALTER TABLE issuance_request
  ADD COLUMN event_id VARCHAR(128) UNIQUE;

-- issueOne()에서
const req = await this.deps.issuanceRepo.create({
  ...
  eventId: event.eventId, // 이벤트 고유 ID
});
// eventId UNIQUE 위반 시 → DB가 자동으로 중복 차단
// ON CONFLICT DO NOTHING + 기존 레코드 반환
```

비교	전략 1	전략 2
방식	애플리케이션 체크	DB UNIQUE 제약
동시성	Race condition 가능	DB가 원자적 보장
구현	findPending() 추가	스키마 변경 필요
필요 조건	이벤트 ID 불필요	이벤트에 고유 ID 있어야

실습에서는 전략 1 사용

두 전략 모두 목표는 같다: "동일 이벤트로 생성된 발행 요청이 이미 있으면 새 요청을 만들지 않는다."

전체 흐름 정리 — 두 레이어의 역할

IssuerService가 조합하는 부품들 — 각자의 책임이 분리되어 있다

레이어	서비스	역할	위치
오케스트레이션	IssuerService	발행 파이프라인 전체 조율	S29
	IssuanceRequestRepository	발행 요청 상태 추적 저장	S29
	IVaspIssuanceClient	VASP에 발행 위탁	S29
정책·조건	IssuancePolicyService	tokenId-amount 조회 (DB 기반)	S28
	EventConditionService	조건 전략 평가 (Strategy 패턴)	S28
지갑	WalletProvisioningService	사용자 지갑 주소 조회	S27

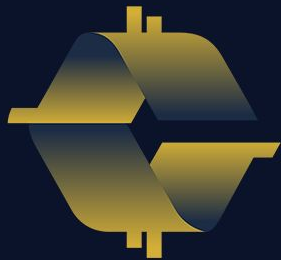
IssuerService는 코드 크기가 가장 작고, 비즈니스 결정은 가장 많이 내린다

각 의존성이 역할을 완전히 캡슐화했기 때문이다. 직접 DB를 건드리지 않는다. 모든 의존성이 인터페이스다.

실습

S29 실습 — 7개 섹션

- [1] 정상 발행 — REQUESTED → SUBMITTED 6단계 흐름
- [2] 조건 미충족 → eligible=false → 레코드 미생성
- [3] 정책 없음 → NoPolicyError → 레코드 미생성
- [4] 지갑 조회 실패 → REQUESTED → FAILED
- [5] VASP 위탁 실패 → FAILED
- [6] 먹등성 — 동일 이벤트 재전송 → 기존 requestId 반환
- [7] Webhook — SUBMITTED → CONFIRMED · 상태 전이 불변



COINCRAFT
ACADEMY

[1] 정상 발행 — REQUESTED → SUBMITTED 6단계 흐름

npm run exercise:s29:1 · issueOne() 전체 파이프라인 정상 경로

```
$ npm run exercise:s29:1
```

실행 코드

```
const repo = new InMemoryIssuanceRepo();
const deps = makeDeps(); // 전부 성공 mock
const svc = new IssuerService({ ...deps, issuanceRepo: repo });

const result = await svc.issueOne({
  userId: 'K-29001',
  eventType: 'WALK_GOAL_MET',
  data: { steps: 12_000 }
});

const req = await repo.findById(result.requestId);
console.log('최종 상태:', req?.status);
console.log('VASP 호출 횟수:', deps.vaspCalls.length);
```

핵심 포인트

6단계 파이프라인 순서:

① 정책 → ② 조건 → ③ 역등성
→ ④ 생성 → ⑤ 지갑 → ⑥ VASP
순서 역전 불가 (각 단계가 다음 단계 전제)

CONFIRMED는 issueOne()에서 설정 안 함:

SUBMITTED까지만 → VASP Webhook이 CONFIRMED 설정
비동기 경로 완전 분리 (섹션 [7] 확인)

makeDeps() mock:

모든 의존성을 성공하는 stub으로 교체
override 파라미터로 실패 주입 (섹션 [2]~[5])

[2] 조건 미충족 → eligible=false → 레코드 미생성

npm run exercise:s29:2 · ②에서 false → ③④⑤⑥ 전부 실행 안 함

```
$ npm run exercise:s29:2
```

실행 코드

```
const repo = new InMemoryIssuanceRepo();
const deps = makeDeps({ eligible: false });
const svc = new IssuerService(
  { ...deps, issuanceRepo: repo });

const result = await svc.issueOne({
  userId: 'K-29002',
  eventType: 'WALK_GOAL_MET',
  data: { steps: 100 }
});

console.log('eligible:', result.eligible);
console.log('requestId:', result.requestId);
console.log('DB 레코드 수:', repo.count());
```

핵심 포인트

② 조건 판단이 가장 빠른 게이트키퍼:

eligible=false → ③④⑤⑥ 전부 실행 안 함

DB 레코드조차 생성 안 함 → 오발행 원천 차단

requestId = "" (빈 문자열):

요청 자체가 없음 = 빈 requestId

호출자는 result.eligible 먼저 확인

S28 EventConditionService와 연동:

conditionService.evaluate() → eligible 반환

false positive 방지 = 온체인 발행 취소 불가

[3] 정책 없음 → NoPolicyError → 레코드 미생성

npm run exercise:s29:3 · ①에서 throw → ②③④⑤⑥ 전부 실행 안 함

```
$ npm run exercise:s29:3
```

실행 코드

```
const repo = new InMemoryIssuanceRepo();
const deps = makeDeps({
  noPolicyFor: 'UNKNOWN_EVENT'
});
const svc = new IssuerService(
  { ...deps, issuanceRepo: repo });

try {
  await svc.issueOne({
    userId: 'K-29003',
    eventType: 'UNKNOWN_EVENT', data: {}
  });
} catch (err) {
  console.log('에러:', err.constructor.name,
    err.message);
}

console.log('DB 레코드 수:', repo.count());
```

핵심 포인트

① 정책 조회가 첫 번째 가드:

NoPolicyError throw → ②③④⑤⑥ 전부 실행 안 함
FAILED 레코드조작 생성 안 함 (요청 미도달)

섹션 [2]와 차이:

[2] eligible=false → 정상 반환 (throw 아님)
[3] NoPolicyError → throw (상위 전파)

정책 관리의 중요성:

issuance_policies 테이블 미등록
= 해당 이벤트 발행 완전 차단
S28 IssuancePolicyService.registerPolicy() 선행 필요

[4] 지갑 조회 실패 → REQUESTED → FAILED 상태 전이

npm run exercise:s29:4 · ⑤ walletFail → REQUESTED → FAILED

```
$ npm run exercise:s29:4
```

실행 코드

```
const repo = new InMemoryIssuanceRepo();
const deps = makeDeps({ walletFail: true });
const svc = new IssuerService(
  { ...deps, issuanceRepo: repo });

try {
  await svc.issueOne({
    userId: 'K-29004',
    eventType: 'WALK_GOAL_MET',
    data: { steps: 12_000 }
  });
} catch (err) {
  console.log('에러:', err.message);
}

// 생성된 레코드 확인
const req = allEntries[0];
console.log('DB 상태:', req?.status);
console.log('walletAddr:', req?.walletAddr);
```

핵심 포인트

④ REQUESTED → ⑤ 실패 → FAILED:

try/catch: 에러 catch → updateStatus(FAILED) → re-throw
레코드는 생성됨 (REQUESTED 상태로) → FAILED 전이

walletAddr=null 유지:

지갑 조회 실패 → setWalletAddr 호출 안 함
failReason에 에러 메시지 기록

[3]과 [4] 차이:

- [3] NoPolicyError: 레코드 미생성 (DB 접근조차 없음)
- [4] walletFail: 레코드 생성 후 FAILED 전이

[5] VASP 위탁 실패 → FAILED

npm run exercise:s29:5 · ⑥ vaspFail → FAILED · walletAddr은 기록됨

```
$ npm run exercise:s29:5
```

실행 코드

```
const repo = new InMemoryIssuanceRepo();
const deps = makeDeps({ vaspFail: true });
const svc = new IssuerService(
  { ...deps, issuanceRepo: repo });

try {
  await svc.issueOne({
    userId: 'K-29005',
    eventType: 'WALK_GOAL_MET',
    data: { steps: 12_000 }
  });
} catch (err) {
  console.log('에러:', err.message);
}

const req = allEntries[0];
console.log('DB 상태:', req?.status);
console.log('walletAddr:', req?.walletAddr);
```

핵심 포인트

- ⑤ 성공 → walletAddr DB에 기록됨:
- ⑥ 실패해도 walletAddr은 남아있음
재시도 시: walletAddr 재조회 불필요

[4]와 [5] 차이:

- [4] walletFail: walletAddr=null
- [5] vaspFail: walletAddr=0xWALLET... (설정됨)

VASP 타임아웃 처리:

네트워크 문제 → FAILED 기록 + re-throw
SUBMITTED 안 됨 → 온체인 미제출 보장
(Webhook이 없으면 CONFIRMED 안 됨)

[6] 멍등성 — 동일 이벤트 재전송 → 기존 requestId 반환

npm run exercise:s29:6 · ③ findPending → 기존 요청 → 새 생성 안 함

```
$ npm run exercise:s29:6
```

실행 코드

```
const repo = new InMemoryIssuanceRepo();
const deps = makeDeps();
const svc = new IssuerService(
  { ...deps, issuanceRepo: repo });

const r1 = await svc.issueOne({
  userId: 'K-29006',
  eventType: 'WALK_GOAL_MET',
  data: { steps: 12_000 }
});
console.log('1차 requestId:', r1.requestId);

const r2 = await svc.issueOne({
  userId: 'K-29006', // 동일 요청
  eventType: 'WALK_GOAL_MET',
  data: { steps: 12_000 }
});
console.log('2차 requestId:', r2.requestId);
console.log('동일:', r1.requestId === r2.requestId);
console.log('DB 레코드:', repo.count());
console.log('VASP 호출:', deps.vaspCalls.length);
```

핵심 포인트

findPending() 키: `userId + eventType + tokenId`:

세 값 모두 일치 + REQUESTED/SUBMITTED 상태 → 기존 반환
CONFIRMED/FAILED는 찾지 않음 → 재발행 허용

멍등성이 필요한 이유:

네트워크 재시도, 메시지 큐 중복 전달
→ 동일 이벤트 중복 도달 시 이중 발행 방지

2차 호출에서 ④⑤⑥ 전부 skip:

VASP 호출 횟수 = 1 (2차에 재호출 없음)
DB 레코드 = 1 (중복 생성 없음)

[7] Webhook — SUBMITTED → CONFIRMED · 상태 전이 불변

npm run exercise:s29:7 · handleVaspWebhook() · CONFIRMED 후 추가 전이 차단

```
$ npm run exercise:s29:7
```

실행 코드

```
const { requestId } = await svc.issueOne({ ... });
console.log('issueOne 완료 status: SUBMITTED');

// VASP Webhook → CONFIRMED
await svc.handleVaspWebhook({
  requestId,
  txHash: '0xdeadbeef',
  status: 'confirmed'
});
const confirmed = await repo.findById(requestId);
console.log('상태:', confirmed?.status);

// 중단 상태 → 추가 전이 불가
try {
  await svc.handleVaspWebhook({
    requestId, status: 'failed'
  });
} catch (err) {
  console.log('불변 조건:', err.message);
}
```

핵심 포인트

issueOne()과 분리된 Webhook 경로:

VASP가 비동기로 Webhook 전송 → CONFIRMED 설정
issueOne()은 SUBMITTED까지만 — 경로 완전 분리

종단 상태 불변 조건:

CONFIRMED/FAILED → updateStatus() throw
온체인 기록 보호 — 이미 체결된 TX 덮어쓰기 방지

txHash 기록:

CONFIRMED 시 txHash 함께 저장
블록체인 트랜잭션 해시 = 온체인 증거

S29 · ISSUERSERVICE — ISSUEONE() 6단계 파이프라인

Thank You

IssuerService · issueOne() · 6단계 파이프라인
먹등성 · 상태 전이 불변 · VASP Webhook · CONFIRMED

M5 · S29

실습 완료

kyobo-digital-asset-platform

COINCRAFT
ACADEMY